

Chapter: SQL String Functions

INITCAP()

INITCAP() converts a string into Title Case.

Title Case means:

- The first letter of each word becomes uppercase.
- All remaining letters become lowercase.

Example

aadya raj becomes Aadya Raj

Another example

HELLO WORLD becomes Hello World

Unlike UPPER() or LOWER(), INITCAP() changes both uppercase and lowercase letters to produce properly formatted words.

Syntax

PostgreSQL

```
SELECT INITCAP(column_name)
FROM Students;
```

Vendor Support

This is the most important part of this chapter.

Database	Supports INITCAP()?
PostgreSQL	✓ Yes
Oracle	✓ Yes
MySQL	✗ No
SQL Server	✗ No
SQLite	✗ No

MySQL has no built-in INITCAP().

For a single-word string, you can simulate it using:

```
SELECT CONCAT(
    UPPER( LEFT( Name, 1 ) ),
    LOWER( SUBSTRING( Name, 2 ) )
)
FROM Students;
```

TRIM()

TRIM() removes unwanted characters from the beginning and/or end of a string.

By default, it removes spaces.

Example

' Aadya ' becomes 'Aadya'

Notice that spaces inside the string are not removed.

'Aadya Raj' remains 'Aadya Raj'

```
SELECT *
FROM Students
WHERE Name = 'Aadya';
```

may not match ' Aadya '

Using

```
TRIM(Name)
```

solves this problem.

' Hello '

TRIM scans:

- left side
- right side

and removes removable characters.

Result

'Hello'

It does not modify the stored data unless used inside an UPDATE.

```
SELECT Name,  
TRIM(Name)  
FROM Students;
```

Original	Trimmed
Aadya	Aadya
Aadya	Aadya
Aadya	Aadya

Removing Specific Characters

TRIM can remove characters other than spaces.

Example

```
SELECT TRIM('*' FROM '***SQL***');
```

Result

SQL

Example

```
SELECT TRIM('-' FROM '---Hello---');
```

Result

Hello

TRIM only removes characters at the edges.

Example

AB*C*D

Using

```
TRIM('*')
```

Result

AB*C*D

Nothing changes.

LEADING

Removes characters only from the beginning.

Example

```
SELECT TRIM(LEADING '*' FROM '***SQL***');
```

Result

```
SQL***
```

```
SELECT TRIM(TRAILING '*' FROM '***SQL***');
```

Result

```
***SQL
```

Best Practices

- Trim user input before inserting into the database.
- Store clean data instead of trimming every query.
- Use TRIM() mainly for presentation or cleanup.

LTRIM()

Trailing spaces remain

```
SELECT LTRIM(' SQL ');
```

Result

```
SQL
```

LTRIM() removes characters from the left (beginning) of a string.

Historically, it removed only spaces, but support for removing arbitrary characters varies by database.

PostgreSQL Character Removal

```
SELECT LTRIM('***SQL', '*');
```

Result

```
SQL
```

RTRIM()

RTRIM() removes characters from the right (end) of a string.

7.2 Syntax

Basic syntax

```
RTRIM(string)
```

PostgreSQL also supports

```
RTRIM(string, characters)
```

PostgreSQL Character Removal

```
SELECT RTRIM('SQL***', '*');
```

Result

```
SQL
```

REPLACE()

REPLACE() searches for a specified substring within a string and replaces every occurrence of that substring with another substring.

Think of it as SQL's global Find and Replace.

Real-world databases often contain inconsistent data.

Examples:

Type	Original Data	Transformation	Clean Data
Phone numbers	987-654-3210	↓	9876543210
Currency	₹25,000	↓	25000
Emails	AADYA@GMAIL.COM	↓	aadya@gmail.com

```
REPLACE(string, old_substring, new_substring)
```

Example

```
SELECT REPLACE('Hello SQL', 'SQL', 'MySQL');
```

Result

Hello MySQL

```
SELECT REPLACE('Aadya', 'a', '*');
```

Result

A*d*y*

Only lowercase **a** is replaced.

Remove Characters

One of the most useful applications.

Simply replace with an empty string.

Example

```
SELECT REPLACE('987-654-3210', '-', '')  
);
```

Result

```
9876543210
```

Nested REPLACE()

Need multiple replacements?

Simply nest them.

Example

```
SELECT REPLACE(  
    REPLACE(  
        '₹25,000',  
        '₹',  
        ''),  
    ',',  
    '');  
);
```

Result

```
25000
```

NULL Behavior

Suppose


```
SELECT REPLACE(NULL, 'a', 'b');
```

Result

NULL

Like most SQL string functions, if any required argument is NULL, the result is generally NULL.

A very common interview pattern is cleaning imported values using nested string functions. For example, converting:

₹ 1,25,000 into 125000 can be done as:

```
SELECT TRIM(
  REPLACE(
    REPLACE(
      Salary,
      '₹',
      ''
    ),
    ',',
    ''
  )
) AS CleanSalary;
```